

Lista de recursão e recorrências

1- Seja f a função definida sobre os inteiros positivos pela recorrência

$$f(n) = \begin{cases} f(1) = 1 \\ f(n) = f(n-1) + 3n + 1 \end{cases} \text{ para } n \geq 2.$$

Resolva a recorrência exatamente.

2- Seja f a função definida pela recorrência

$$f(n) = \begin{cases} f(1) = 1 \\ f(2) = 2 \\ f(n) = f(n-2) + 2 \end{cases} \text{ para } n \geq 3.$$

Resolva a recorrência exatamente.

3- Suponha que $f(1) = 1$ e $f(n) = f(n-1) + 2n$ para $n = 2, 3, 4, \dots$. Mostre que $f(n) = \Omega(n^2)$.

4- Seja $T(n)$ uma função definida sobre $\{0, 1, 2, \dots\}$. Suponha que existem $\alpha > 0$ e $\beta > 0$ tais que $T(1) \leq \alpha$ e $T(n) \leq T(n-k) + \beta k$ para todo $n > 1$. Mostre que $T(n) \leq \max(\alpha, \beta)n$.

5- Seja f a função definida sobre inteiros positivos pela recorrência

$$f(n) = \begin{cases} f(1) = 1 \\ f(n) = f\left[\frac{n}{2}\right] + f\left[\frac{n}{2}\right] + 6n + 7 \end{cases} \text{ para } n \geq 2.$$

Verifique que a recorrência é honesta, ou seja, de fato define uma função sobre os inteiros positivos. Resolva a recorrência. Não é preciso resolver a recorrência de maneira exata: basta mostrar que $f(n) = O(g(n))$ para alguma função g razoavelmente pequena.

6- Escreva um algoritmo que remova todos os elementos nulos de um vetor $A[p \dots r]$ mas preserve a ordem relativa dos demais elementos. Faça duas versões: uma iterativa e uma recursiva. Repita o problema depois de substituir o vetor por uma lista encadeada: o seu algoritmo deverá aceitar como entrada o endereço do primeiro nó da lista.

7- Esta questão envolve retângulos cuja altura e largura são inteiros positivos. Um retângulo é elementar se tem altura 1 ou largura 1. Vamos considerar partições de um retângulo R em retângulos elementares. Seja $p(m, n)$ o número de diferentes partições de um retângulo de altura m e largura n . Por exemplo, $p(2, 2) = 7$.

- (a) Escreva uma recorrência para $p(1, n)$.
- (b) Resolva a recorrência do item 1.
- (c) Escreva uma recorrência para $p(2, n)$.
- (d) Descreva um algoritmo para determinar $p(2, n)$.
- (e) Determine a complexidade de seu algoritmo.

8- Suponha que um vetor $A[1 \dots n]$ tem exatamente m elementos não-nulos e que os índices desses elementos são $i_1 < i_2 < \dots < i_m$. Escreva um algoritmo recursivo eficiente que tenha o mesmo efeito que a sequência de atribuições

$$A[1] \leftarrow A[i_1], A[2] \leftarrow A[i_2], \dots, A[m] \leftarrow A[i_m].$$

O seu algoritmo deve receber apenas n e $A[1 \dots n]$ e devolver m .

9- Seja T a função definida sobre os inteiros positivos pela recorrência

$$T(n) = \begin{cases} T(1) = c \\ T(n) = 3T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n & \text{para } n \geq 2, \end{cases}$$

onde c é uma constante. A que classe Θ a função pertence?

10- Seja T a função definida sobre os inteiros positivos pela recorrência

$$T(n) = \begin{cases} T(1) = 1 \\ T(n) \leq T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(1 + \left\lceil \frac{n}{2} \right\rceil\right) + \beta n & \text{para } n \geq 2, \end{cases}$$

sendo β uma constante positiva. A que classe O a função pertence?

11- Considere a sequência de Fibonacci, em que cada número na sequência é a soma dos dois anteriores. Ou seja, a sequência começa com 0 e 1, e cada número subsequente é a soma dos dois números anteriores. Por exemplo, os primeiros números na sequência são: 0, 1, 1, 2, 3, 5, 8, 13, ...

(a) Escreva uma função recursiva em uma linguagem de programação de sua escolha para calcular o n -ésimo número na sequência de Fibonacci.

(b) Determine a recorrência para a função que você escreveu no item 1.

(c) Explique a complexidade de tempo da sua função recursiva em termos de n .

(d) Escreva uma versão iterativa da função para calcular o n -ésimo número na sequência de Fibonacci e compare sua complexidade de tempo com a versão recursiva.

12- Considere o algoritmo de ordenação por inserção, que consiste em inserir elementos de uma lista desordenada em uma sublista ordenada, de maneira que a sublista ordenada cresça até englobar todos os elementos da lista original.

(a) Descreva o funcionamento do algoritmo de ordenação por inserção.

(b) Implemente uma versão recursiva do algoritmo de ordenação por inserção em uma linguagem de programação de sua escolha.

(c) Analise a complexidade de tempo do algoritmo de ordenação por inserção recursivo no melhor e pior caso.

(d) Compare a complexidade de tempo do algoritmo de ordenação por inserção com outros algoritmos de ordenação conhecidos, como o algoritmo de seleção e o algoritmo de merge sort.

(e) Explique uma situação em que o algoritmo de ordenação por inserção seria preferível em relação aos outros algoritmos de ordenação.